

=====

BIBLE PSUTIL – RÉFÉRENCE EXHAUSTIVE DU MODULE

LÉGENDE : [sys] = système | [proc] = Process | [cpu] = CPU | [mem] = mémoire
[disk] = disque | [net] = réseau | [sens] = capteurs

=====

[cpu] CPU – cpu_percent / cpu_times / cpu_freq / cpu_count
--

DESCRIPTION : Mesurer l'utilisation, la fréquence et les caractéristiques du CPU.
POURQUOI : Monitoring de performance, détection de surcharge, profiling système.
CONTEXTE : Scripts de supervision, dashboards, alertes automatiques.

— UTILISATION —

```
import psutil
```

psutil.cpu_percent()	Usage global en % (non bloquant, ~0%)
psutil.cpu_percent(interval=1)	Usage sur 1 seconde (bloquant, fiable)
psutil.cpu_percent(interval=0.5)	Sur 0.5 secondes
psutil.cpu_percent(percpu=True)	Usage par cœur → liste [%core0, %core1...]
psutil.cpu_percent(interval=1, percpu=True)	Par cœur sur 1 seconde

— TEMPS CPU —

psutil.cpu_times()	Temps CPU global (scputimes)
psutil.cpu_times().user	Temps en mode utilisateur (s)
psutil.cpu_times().system	Temps en mode noyau (s)
psutil.cpu_times().idle	Temps inactif (s)
psutil.cpu_times().iowait	Attente I/O (Linux)
psutil.cpu_times().irq	Interruptions matérielles (Linux)
psutil.cpu_times().softirq	Interruptions logicielles (Linux)
psutil.cpu_times().steal	Vol CPU (VM, Linux)
psutil.cpu_times().nice	Processus nice (Linux/macOS)
psutil.cpu_times(percpu=True)	Par cœur → liste de scputimes
psutil.cpu_times_percent()	Pourcentage de chaque état CPU
psutil.cpu_times_percent(interval=1)	Sur 1 seconde
psutil.cpu_times_percent(percpu=True)	Par cœur

— FRÉQUENCE —

psutil.cpu_freq()	Fréquence CPU (scpu_freq)
psutil.cpu_freq().current	Fréquence actuelle (MHz)
psutil.cpu_freq().min	Fréquence minimale (MHz)
psutil.cpu_freq().max	Fréquence maximale (MHz)
psutil.cpu_freq(percpu=True)	Par cœur → liste

— NOMBRE DE CŒURS —

psutil.cpu_count()	Cœurs logiques (avec hyperthreading)
psutil.cpu_count(logical=True)	Cœurs logiques (défaut)
psutil.cpu_count(logical=False)	Cœurs physiques
psutil.cpu_stats()	Statistiques CPU (scpu_stats)
psutil.cpu_stats().ctx_switches	Changements de contexte
psutil.cpu_stats().interrupts	Interruptions matérielles
psutil.cpu_stats().soft_interrupts	Interruptions logicielles
psutil.cpu_stats().syscalls	Appels système (macOS/Windows)

— CHARGE SYSTÈME —

psutil.getloadavg()	Charge moy. sur 1, 5, 15 min (Unix)
load1, load5, load15 = psutil.getloadavg()	

EXEMPLES :

```
# Utilisation CPU toutes les secondes
import time
while True:
    usage = psutil.cpu_percent(interval=1)
    par_coeur = psutil.cpu_percent(percpu=True)
    print(f"CPU global: {usage}% | Cœurs: {par_coeur}")
    time.sleep(0) # interval déjà attendu

# Alerte si surcharge
if psutil.cpu_percent(interval=1) > 90:
    print("⚠ CPU > 90% !")
```

[mem] Mémoire – virtual_memory / swap_memory
--

DESCRIPTION : Surveiller l'utilisation de la RAM et du swap.
 POURQUOI : Détecter les fuites mémoire, prévenir les OOM (Out of Memory).

— MÉMOIRE VIRTUELLE (RAM)

mem = psutil.virtual_memory()	svmem namedtuple
mem.total	RAM totale (bytes)
mem.available	RAM disponible (bytes)
mem.used	RAM utilisée (bytes)
mem.free	RAM libre (bytes, pas forcément dispo)
mem.percent	Pourcentage utilisé (0-100)
mem.active	Mémoire active (Linux/macOS)
mem.inactive	Mémoire inactive (Linux/macOS)
mem.buffers	Buffers noyau (Linux)
mem.cached	Cache disque (Linux/macOS)
mem.shared	Mémoire partagée (Linux)
mem.slab	Slab allocator (Linux)
mem.wired	Mémoire câblée (macOS)

— MÉMOIRE SWAP

swap = psutil.swap_memory()	sswap namedtuple
swap.total	Swap total (bytes)
swap.used	Swap utilisé (bytes)
swap.free	Swap libre (bytes)
swap.percent	Pourcentage utilisé (0-100)
swap.sin	Octets swappés depuis disque
swap.sout	Octets swappés vers disque

— CONVERSION

```
def octets_vers_lisible(n):
    for unite in ["o", "Ko", "Mo", "Go", "To"]:
        if n < 1024: return f"{n:.1f} {unite}"
        n /= 1024
```

EXEMPLES :

```
mem = psutil.virtual_memory()
print(f"RAM : {mem.percent}% utilisé")
print(f"  Total      : {mem.total / 1024**3:.1f} Go")
print(f"  Disponible: {mem.available / 1024**3:.1f} Go")
print(f"  Utilisé     : {mem.used / 1024**3:.1f} Go")

swap = psutil.swap_memory()
if swap.percent > 50:
    print(f"⚠ Swap {swap.percent}% utilisé !")
```

[disk] Disques – disk_partitions / disk_usage / disk_io_counters

DESCRIPTION : Inspecter les partitions, l'espace disque et les I/O.
 POURQUOI : Surveiller l'espace, détecter des goulots d'étranglement I/O.

— PARTITIONS

psutil.disk_partitions()	Partitions montées → liste de sdiskpart
psutil.disk_partitions(all=False)	Seulement les physiques (défaut)
psutil.disk_partitions(all=True)	Toutes (y compris /proc, tmpfs...)
for part in psutil.disk_partitions():	
part.device	Périphérique (/dev/sda1, C:\...)
part.mountpoint	Point de montage (/ , /home, C:\...)
part.fstype	Système de fichiers (ext4, ntfs...)
part.opts	Options de montage (rw, ro...)
part.maxfile	Longueur max de nom de fichier
part.maxpath	Longueur max de chemin

— UTILISATION

usage = psutil.disk_usage("/")	Usage de / (sdiskusage)
usage = psutil.disk_usage("C:\\")	Windows
usage.total	Taille totale (bytes)
usage.used	Utilisé (bytes)
usage.free	Libre (bytes)
usage.percent	Pourcentage utilisé (0-100)

— COMPTEURS I/O

psutil.disk_io_counters()	Totaux I/O (sdiskio)
---------------------------	----------------------

psutil.disk_io_counters(perdisk=True)	Par disque → dict {nom: sdiskio}
psutil.disk_io_counters(nowrap=True)	Gère les remises à zéro du compteur
io = psutil.disk_io_counters()	
io.read_count	Nombre de lectures
io.write_count	Nombre d'écritures
io.read_bytes	Octets lus
io.write_bytes	Octets écrits
io.read_time	Temps passé à lire (ms)
io.write_time	Temps passé à écrire (ms)
io.busy_time	Temps d'activité (ms, Linux)

```

EXEMPLES      :
# Espace de toutes les partitions
for part in psutil.disk_partitions():
    try:
        u = psutil.disk_usage(part.mountpoint)
        print(f"{part.mountpoint}: {u.percent}% "
              f"({u.free/1024**3:.1f} Go libres)")
    except PermissionError:
        pass

# Vitesse I/O (mesure sur 1 seconde)
import time
avant = psutil.disk_io_counters()
time.sleep(1)
apres = psutil.disk_io_counters()
print(f"Lecture : {(apres.read_bytes - avant.read_bytes)/1024:.0f} Ko/s")
print(f"Écriture: {(apres.write_bytes - avant.write_bytes)/1024:.0f} Ko/s")

```

[net] Réseau – net_io_counters / net_connections / net_if_addrs

DESCRIPTION : Surveiller les interfaces réseau, connexions et trafic.
POURQUOI : Monitoring réseau, détection d'activité suspecte, bande passante.

— COMPTEURS I/O RÉSEAU —

psutil.net_io_counters()	Totaux réseau (snetio)
psutil.net_io_counters(pernic=True)	Par interface → dict {nom: snetio}
psutil.net_io_counters(nowrap=True)	Gère les remises à zéro
io = psutil.net_io_counters()	
io.bytes_sent	Octets envoyés
io.bytes_recv	Octets reçus
io.packets_sent	Paquets envoyés
io.packets_recv	Paquets reçus
io.errin	Erreurs en réception
io.errout	Erreurs en émission
io.dropin	Paquets reçus perdus
io.dropout	Paquets émis perdus

— INTERFACES RÉSEAU —

psutil.net_if_addrs()	Adresses par interface → dict
psutil.net_if_stats()	Stats par interface → dict
addrs = psutil.net_if_addrs()	
for iface, addreses in addrs.items():	
for addr in addreses:	
addr.family	AF_INET, AF_INET6, AF_LINK...
addr.address	Adresse IP ou MAC
addr.netmask	Masque de sous-réseau
addr.broadcast	Adresse de broadcast
addr.ptp	Point-à-point (PPP)
stats = psutil.net_if_stats()	
for iface, stat in stats.items():	
stat.isup	True si interface active
stat.duplex	Mode duplex (NIC_DUPLEX_FULL...)
stat.speed	Vitesse en Mbps
stat.mtu	MTU en octets

— CONNEXIONS RÉSEAU —

psutil.net_connections()	Toutes les connexions
psutil.net_connections(kind="inet")	TCP + UDP IPv4/IPv6

psutil.net_connections(kind="inet4")	IPv4 seulement
psutil.net_connections(kind="inet6")	IPv6 seulement
psutil.net_connections(kind="tcp")	TCP seulement
psutil.net_connections(kind="tcp4")	TCP IPv4
psutil.net_connections(kind="udp")	UDP seulement
psutil.net_connections(kind="unix")	Sockets Unix


```

for conn in psutil.net_connections(kind="tcp"):
    conn.fd          Descripteur de fichier
    conn.family      AF_INET, AF_INET6...
    conn.type        SOCK_STREAM, SOCK_DGRAM...
    conn.laddr       Adresse locale (ip, port)
    conn.raddr       Adresse distante (ip, port)
    conn.status      État TCP (ESTABLISHED, LISTEN...)
    conn.pid         PID du processus propriétaire

```

```

EXEMPLES      :
# Bande passante en temps réel
import time
avant = psutil.net_io_counters()
time.sleep(1)
apres = psutil.net_io_counters()
envoi   = (apres.bytes_sent - avant.bytes_sent) / 1024
reception = (apres.bytes_recv - avant.bytes_recv) / 1024
print(f"↑ {envoi:.1f} Ko/s   ↓ {reception:.1f} Ko/s")

# Connexions actives
conns = [c for c in psutil.net_connections(kind="tcp")
         if c.status == "ESTABLISHED"]
print(f"{len(conns)} connexions TCP établies")

# Adresse IP locale
addrs = psutil.net_if_addrs()
for iface, liste in addrs.items():
    for addr in liste:
        if addr.family == 2: # AF_INET
            print(f"{iface}: {addr.address}")

```

[sens] Capteurs – sensors_temperatures / sensors_fans / sensors_battery |

DESCRIPTION : Lire les températures, vitesses de ventilateurs et état de la batterie.
POURQUOI : Monitoring thermique, protection contre la surchauffe.
CONTEXTE : Linux et macOS principalement. Partiel sur Windows.

— TEMPÉRATURES —

```

psutil.sensors_temperatures()          Dict {composant: [shwtemp...]}
psutil.sensors_temperatures(fahrenheit=False) En Celsius (défaut)
psutil.sensors_temperatures(fahrenheit=True)  En Fahrenheit

for composant, capteurs in psutil.sensors_temperatures().items():
    for capteur in capteurs:
        capteur.label          Nom du capteur
        capteur.current         Température actuelle (°C)
        capteur.high            Seuil haut (°C)
        capteur.critical        Seuil critique (°C)

```

— VENTILATEURS —

```

psutil.sensors_fans()          Dict {composant: [sfan...]}
for composant, fans in psutil.sensors_fans().items():
    for fan in fans:
        fan.label              Nom du ventilateur
        fan.current            Vitesse actuelle (RPM)

```

— BATTERIE —

```

psutil.sensors_battery()       sbattery ou None si pas de batterie
bat = psutil.sensors_battery()
bat.percent                    Niveau en % (0-100)
bat.secsleft                   Secondes restantes (ou POWER_TIME_*)
bat.power_plugged              True si branché secteur

psutil.POWER_TIME_UNLIMITED    Branché (temps illimité)
psutil.POWER_TIME_UNKNOWN      Temps inconnu

```

```

EXEMPLES      :
# Alertes thermiques
temps = psutil.sensors_temperatures()
for composant, capteurs in temps.items():
    for c in capteurs:
        if c.current and c.critical:
            if c.current > c.critical * 0.9:
                print(f"⚠ {composant}/{c.label}: "
                    f"{c.current:.1f}°C proche du critique ({c.critical}°C)")

# État batterie
bat = psutil.sensors_battery()
if bat:
    statut = "secteur" if bat.power_plugged else "batterie"
    print(f"Batterie : {bat.percent:.0f}% ({statut})")
    if not bat.power_plugged and bat.secsleft != psutil.POWER_TIME_UNKNOWN:
        heures = bat.secsleft // 3600
        minutes = (bat.secsleft % 3600) // 60
        print(f"Autonomie restante : {heures}h{minutes:02d}m")

```

[proc] Processus – Process / pids / process_iter
--

DESCRIPTION : Lister, inspecter et contrôler les processus du système.
POURQUOI : Supervision, kill, monitoring de ressources par processus.

— LISTER LES PROCESSUS —

psutil.pids()	Liste de tous les PIDs
psutil.process_iter()	Itérateur de Process
psutil.process_iter(["pid", "name", "cpu_percent"])	Avec attributs pré-chargés
psutil.pid_exists(1234)	True si le PID existe

— CRÉER UN OBJET PROCESS —

p = psutil.Process()	Processus courant
p = psutil.Process(pid=1234)	Processus par PID
p = psutil.Process(os.getpid())	Processus Python courant

— INFORMATIONS GÉNÉRALES —

p.pid	PID
p.ppid()	PID du parent
p.name()	Nom de l'exécutable
p.exe()	Chemin complet de l'exécutable
p.cmdline()	Ligne de commande complète (liste)
p.cwd()	Répertoire de travail courant
p.status()	État : running, sleeping, zombie...
p.create_time()	Timestamp de création (Unix)
p.username()	Utilisateur propriétaire
p.uids()	UIDs (réel, effectif, sauvegardé) Unix
p.gids()	GIDs Unix
p.terminal()	Terminal associé (Unix)
p.environ()	Variables d'environnement (dict)
p.is_running()	True si le processus tourne encore
p.parent()	Process parent
p.parents()	Liste de tous les ancêtres
p.children()	Processus enfants directs
p.children(recursive=True)	Tous les descendants

— RESSOURCES CPU —

p.cpu_percent()	Usage CPU % (non bloquant, 1er appel=0)
p.cpu_percent(interval=1)	Sur 1 seconde
p.cpu_times()	Temps CPU du processus (pcputimes)
p.cpu_times().user	Temps utilisateur (s)
p.cpu_times().system	Temps système (s)
p.cpu_times().children_user	Temps user des enfants (Linux)
p.cpu_times().children_system	Temps system des enfants (Linux)
p.cpu_times().iowait	Attente I/O (Linux)
p.cpu_affinity()	Liste des cœurs autorisés
p.cpu_affinity([0, 1])	Restreindre aux cœurs 0 et 1
p.cpu_num()	Cœur sur lequel tourne le process
p.num_threads()	Nombre de threads
p.threads()	Liste des threads (pthread)
p.num_ctx_switches()	Changements de contexte

— RESSOURCES MÉMOIRE —

p.memory_info()	Infos mémoire (pmem)
p.memory_info().rss	RSS : mémoire physique réelle (bytes)
p.memory_info().vms	VMS : mémoire virtuelle totale (bytes)
p.memory_info().shared	Mémoire partagée (Linux)
p.memory_info().text	Code (Linux)
p.memory_info().data	Données (Linux)
p.memory_info().lib	Librairies (Linux)
p.memory_full_info()	Infos complètes (uss, pss...)
p.memory_full_info().uss	USS : mémoire unique au processus
p.memory_full_info().pss	PSS : proportionnelle partagée
p.memory_percent()	% de la RAM totale
p.memory_percent(memtype="rss")	Par type (rss, vms, uss...)
p.memory_maps()	Mappings mémoire
p.memory_maps(grouped=True)	Groupés par chemin

— FICHIERS ET RÉSEAU —

p.open_files()	Fichiers ouverts → liste de popenfile
p.connections()	Connexions réseau du processus
p.connections(kind="tcp")	Connexions TCP
p.net_connections()	Alias de connections() (5.3+)
p.num_fds()	Nombre de fd ouverts (Unix)
p.num_handles()	Nombre de handles (Windows)
p.io_counters()	Compteurs I/O du processus
for f in p.open_files():	
f.path	Chemin du fichier
f.fd	Descripteur de fichier
f.position	Position courante
f.mode	Mode d'ouverture

— PRIORITÉ ET CONTRÔLE —

p.nice()	Valeur nice courante (-20 à 19)
p.nice(10)	Définit la valeur nice
p.ionice()	Priorité I/O (Linux/Windows)
p.ionice(psutil.IOPRIO_CLASS_IDLE)	Priorité I/O basse
p.rlimit(psutil.RLIMIT_NOFILE)	Limite de ressource (Unix)
p.rlimit(psutil.RLIMIT_NOFILE, (100, 200))	Définit soft/hard limit

— ENVOYER DES SIGNAUX —

p.send_signal(signal.SIGTERM)	Envoie un signal
p.terminate()	SIGTERM (arrêt propre)
p.kill()	SIGKILL (arrêt forcé)
p.suspend()	SIGSTOP (pause)
p.resume()	SIGCONT (reprise)
p.wait(timeout=3)	Attend la fin → code retour

— GESTION DES ERREURS —

psutil.NoSuchProcess	PID inexistant ou terminé
psutil.AccessDenied	Permissions insuffisantes
psutil.ZombieProcess	Processus zombie
psutil.TimeoutExpired	Timeout de p.wait()

EXEMPLES :

```
# Top 5 processus par mémoire RSS
procs = []
for p in psutil.process_iter(["pid", "name", "memory_info"]):
    try:
        procs.append(p.info)
    except (psutil.NoSuchProcess, psutil.AccessDenied):
        pass

top5 = sorted(procs,
    key=lambda x: x["memory_info"].rss if x["memory_info"] else 0,
    reverse=True)[:5]
for p in top5:
    rss = p["memory_info"].rss / 1024**2
    print(f"  PID {p['pid']:6d}  {p['name']:<25} {rss:.1f} Mo")

# Top 5 par CPU
for p in psutil.process_iter(["pid", "name"]):
    p.cpu_percent() # Premier appel = initialise
import time; time.sleep(1)
top_cpu = sorted(
    [(p.info["pid"], p.info["name"], p.cpu_percent())
    for p in psutil.process_iter(["pid", "name"])]
```

```
if not any(e for e in [psutil.NoSuchProcess, psutil.AccessDenied]),
key=lambda x: x[2], reverse=True)[:5]
```

```
# Tuer un processus par nom
for p in psutil.process_iter(["name"]):
    if p.info["name"] == "processus_a_tuer":
        p.terminate()
        p.wait(timeout=5)

# Surveiller ses propres ressources
moi = psutil.Process()
print(f"Mon RSS : {moi.memory_info().rss / 1024**2:.1f} Mo")
print(f"Mon CPU : {moi.cpu_percent(interval=1):.1f}%")
```

```
[sys] Informations système – boot_time / users / pids
```

DESCRIPTION : Informations générales sur le système d'exploitation.
POURQUOI : Uptime, utilisateurs connectés, inventaire système.

FONCTIONS PRINCIPALES :

```
psutil.boot_time()          Timestamp de démarrage (Unix)
psutil.users()              Utilisateurs connectés → liste
```

```
import datetime
boot = datetime.datetime.fromtimestamp(psutil.boot_time())
uptime = datetime.datetime.now() - boot
```

```
for user in psutil.users():
    user.name          Nom d'utilisateur
    user.terminal      Terminal (tty, pts...)
    user.host          Hôte distant (SSH)
    user.started       Timestamp de connexion
    user.pid           PID de la session (Unix)
```

EXEMPLES :

```
import datetime
boot_dt = datetime.datetime.fromtimestamp(psutil.boot_time())
uptime = datetime.datetime.now() - boot_dt
jours = uptime.days
heures = uptime.seconds // 3600
minutes = (uptime.seconds % 3600) // 60
print(f"Uptime : {jours}j {heures}h {minutes}m")

print("Utilisateurs connectés :")
for u in psutil.users():
    heure = datetime.datetime.fromtimestamp(u.started)
    print(f" {u.name} sur {u.terminal} depuis {heure:%H:%M}")
```

```
[proc] process_iter avancé – collecte efficace
```

DESCRIPTION : Parcourir tous les processus de manière efficace et sûre.
POURQUOI : Éviter les exceptions sur processus disparus, optimiser la collecte.

PATTERN RECOMMANDÉ :

```
# Pré-charger les attributs voulus (évite appels multiples)
attributs = ["pid", "name", "username", "cpu_percent",
             "memory_info", "status", "create_time"]

for proc in psutil.process_iter(attributs):
    try:
        info = proc.info
        print(info["name"], info["memory_info"].rss)
    except (psutil.NoSuchProcess,
            psutil.AccessDenied,
            psutil.ZombieProcess):
        pass
```

— FILTRES COURANTS —

```
# Processus d'un utilisateur
mon_user = psutil.Process().username()
mes_procs = [p for p in psutil.process_iter(["username", "name"])]
```

```

        if p.info.get("username") == mon_user]

# Processus utilisant plus de X% CPU
gros_cpu = []
for p in psutil.process_iter(["pid", "name", "cpu_percent"]):
    try:
        if p.info["cpu_percent"] > 10:
            gros_cpu.append(p.info)
    except (psutil.NoSuchProcess, psutil.AccessDenied):
        pass

# Chercher un processus par nom
def trouver_par_nom(nom):
    return [p for p in psutil.process_iter(["name"])
            if p.info["name"].lower() == nom.lower()]

# Arbre de processus
def arbre_processus(pid=None):
    if pid is None:
        pid = psutil.Process().pid
    p = psutil.Process(pid)
    enfants = p.children(recursive=True)
    return [p] + enfants

```

[sys] Dashboard de monitoring – recette complète

DESCRIPTION : Collecter toutes les métriques système en une passe.
 POURQUOI : Base pour un système de monitoring ou un dashboard.

RECETTE COMPLÈTE :

```

import psutil, datetime, time

def snapshot_systeme():
    """Collecte un instantané complet du système."""
    cpu = psutil.cpu_percent(interval=1)
    mem = psutil.virtual_memory()
    swap = psutil.swap_memory()
    disk = psutil.disk_usage("/")

    # I/O réseau (delta sur 1s)
    net_avant = psutil.net_io_counters()
    time.sleep(1)
    net_apres = psutil.net_io_counters()

    return {
        "timestamp": datetime.datetime.now().isoformat(),
        "cpu": {
            "percent": cpu,
            "count": psutil.cpu_count(),
            "freq_mhz": psutil.cpu_freq().current if psutil.cpu_freq() else None,
            "load_avg": psutil.getloadavg() if hasattr(psutil, "getloadavg") else None,
        },
        "memory": {
            "total_go": mem.total / 1024**3,
            "used_go": mem.used / 1024**3,
            "percent": mem.percent,
            "swap_pct": swap.percent,
        },
        "disk": {
            "total_go": disk.total / 1024**3,
            "used_go": disk.used / 1024**3,
            "percent": disk.percent,
        },
        "network": {
            "sent_kbps": (net_apres.bytes_sent - net_avant.bytes_sent) / 1024,
            "recv_kbps": (net_apres.bytes_recv - net_avant.bytes_recv) / 1024,
        },
        "processes": len(psutil.pids()),
    }

snap = snapshot_systeme()
print(f"CPU: {snap['cpu']['percent']}% "
      f"RAM: {snap['memory']['percent']}% ")

```



```
f"Disk: {snap['disk']['percent']}%")
```

RÉCAPITULATIF PSUTIL

CATÉGORIE	FONCTION / ATTRIBUT	RETOUR
CPU global	psutil.cpu_percent(interval=1)	float %
CPU par cœur	psutil.cpu_percent(percpu=True)	liste de float
Temps CPU	psutil.cpu_times()	scputimes
Fréquence CPU	psutil.cpu_freq()	scpufreq (MHz)
Nb de cœurs	psutil.cpu_count(logical=False)	int
Charge système	psutil.getloadavg()	(1m, 5m, 15m)
Statistiques CPU	psutil.cpu_stats()	scpustats
RAM	psutil.virtual_memory()	svmem
Swap	psutil.swap_memory()	sswap
Partitions	psutil.disk_partitions()	liste sdiskpart
Espace disque	psutil.disk_usage("/")	sdiskusage
I/O disque	psutil.disk_io_counters()	sdiskio
Trafic réseau	psutil.net_io_counters()	snetio
Interfaces réseau	psutil.net_if_addrs()	dict {iface: [snicaddr]}
Stats interfaces	psutil.net_if_stats()	dict {iface: snicstats}
Connexions TCP/UDP	psutil.net_connections(kind="tcp")	liste sconn
Températures	psutil.sensors_temperatures()	dict
Ventilateurs	psutil.sensors_fans()	dict
Batterie	psutil.sensors_battery()	sbattery
Démarrage	psutil.boot_time()	timestamp float
Utilisateurs	psutil.users()	liste suser
Liste PIDs	psutil.pids()	liste int
Itérer processus	psutil.process_iter(attrs)	itérateur Process
PID existe	psutil.pid_exists(pid)	bool
Accéder Process	psutil.Process(pid)	Process
Nom du process	p.name()	str
Exe du process	p.exe()	str
CPU du process	p.cpu_percent(interval=1)	float %
RAM du process	p.memory_info().rss	bytes
% RAM du process	p.memory_percent()	float
Statut	p.status()	str
Enfants	p.children(recursive=True)	liste Process
Fichiers ouverts	p.open_files()	liste popenfile
Threads	p.num_threads()	int
Tuer proprement	p.terminate() + p.wait(timeout=5)	
Tuer forcement	p.kill()	

— EXCEPTIONS —

psutil.NoSuchProcess	Process disparu entre le listage et l'accès
psutil.AccessDenied	Permissions insuffisantes (process système)
psutil.ZombieProcess	Processus zombie (Unix)
psutil.TimeoutExpired	p.wait() a dépassé le timeout

— COMPATIBILITÉ PLATEFORME —

FONCTIONNALITÉ	LINUX	MACOS	WINDOWS
cpu_percent	☐	☐	☐
cpu_freq	☐	☐	☐
getloadavg	☐	☐	☐
disk_io_counters	☐	☐	☐
net_connections	☐	☐	☐ (partiel)
sensors_temperatures	☐	☐	☐
sensors_fans	☐	☐	☐
sensors_battery	☐	☐	☐
p.cpu_affinity	☐	☐	☐
p.ionice	☐	☐	☐
p.rlimit	☐	☐	☐
p.environ	☐	☐	☐
p.num_fds	☐	☐	☐
p.num_handles	☐	☐	☐